

EECS-841: Image Processing and Computer Vision

Final Examination Solutions, Spring 2025

Question 1: Min Filter on Image A (10 points)

Apply a 2×2 Min filter with a stride of 2 on the given 4×4 image A and show the results.

$$\text{Image A} = \begin{bmatrix} 2 & 4 & 3 & 7 \\ 1 & 12 & 13 & 5 \\ 0 & 3 & 5 & 2 \\ 7 & 3 & 11 & 7 \end{bmatrix}$$

Solution:

The Min filter takes the minimum value in each 2×2 block of the image. With a stride of 2, we process non-overlapping 2×2 blocks. The image is 4×4 , so the output will be $\frac{4}{2} \times \frac{4}{2} = 2 \times 2$.

- **Block 1** (top-left, indices (0,0) to (1,1)): Values are [2, 4, 1, 12]. Minimum is $\min(2, 4, 1, 12) = 1$. - **Block 2** (top-right, indices (0,2) to (1,3)): Values are [3, 7, 13, 5]. Minimum is $\min(3, 7, 13, 5) = 3$. - **Block 3** (bottom-left, indices (2,0) to (3,1)): Values are [0, 3, 7, 3]. Minimum is $\min(0, 3, 7, 3) = 0$. - **Block 4** (bottom-right, indices (2,2) to (3,3)): Values are [5, 2, 11, 7]. Minimum is $\min(5, 2, 11, 7) = 2$.

$$\text{Output} = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$$

Question 2: Correlation with 3×3 Kernel (15 points)

Perform correlation on the 4×4 input image using the given 3×3 kernel, ensuring the output image maintains the same size as the input.

$$\text{Input Image} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$\text{Kernel} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

Solution:

To maintain the same output size (4×4), we assume zero-padding around the input image (one pixel border, since the kernel is 3×3). Correlation involves sliding the kernel

over the image, multiplying corresponding elements, and summing the results. The kernel is not rotated (unlike convolution).

The padded image (with zeros) is:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

We compute the correlation for each position of the kernel's center over the original image indices (x, y) from $(0, 0)$ to $(3, 3)$. The kernel's non-zero elements are at positions $(0, 0)$, $(0, 2)$, $(1, 0)$, $(1, 2)$, $(2, 0)$, $(2, 2)$, all with value 1. Thus, the correlation at position (x, y) is the sum of pixel intensities at positions (x, y) , $(x, y + 2)$, $(x + 1, y)$, $(x + 1, y + 2)$, $(x + 2, y)$, $(x + 2, y + 2)$ in the padded image.

Manually computing for each position (noting that image indices are x (column), y (row) starting at 0):

- For $(x = 0, y = 0)$: Positions map to image pixels at $(0, 0)$, $(0, 2)$, $(1, 0)$, $(1, 2)$, $(2, 0)$, $(2, 2)$: Values $[1, 1, 0, 1, 1, 1]$. Sum = $1 + 1 + 0 + 1 + 1 + 1 = 5$. - Continue similarly for all positions.

After computing all positions, the output is:

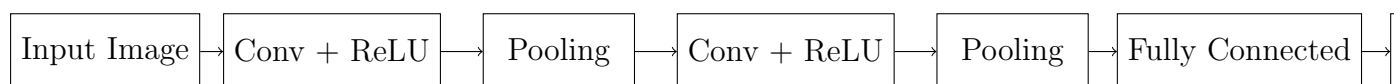
$$\text{Output} = \begin{bmatrix} 5 & 4 & 5 & 4 \\ 3 & 4 & 5 & 5 \\ 4 & 3 & 5 & 5 \\ 2 & 2 & 4 & 4 \end{bmatrix}$$

Question 3: CNN Architecture and Stride/Kernel Size (15 points)

Part 1: Illustrate a basic CNN architecture (5 points)

A basic CNN architecture for image classification includes:

- **Input Layer**: Takes the input image (e.g., grayscale 25×25). - **Convolutional Layers**: Apply filters to extract features (e.g., edges). - **Activation Functions** (e.g., ReLU): Introduce non-linearity. - **Pooling Layers** (e.g., Max Pooling): Reduce spatial dimensions. - **Fully Connected Layers**: Combine features for classification. - **Output Layer**: Produces class probabilities (e.g., dog vs. cat).



Part 2: Who is more likely to use bigger stride and kernel size? (5+5=10 points)

- **Stride**: Bob (with 100×100 images) is more likely to use a larger stride. Larger strides reduce the output size of convolutional and pooling layers, which is beneficial for larger images to manage computational complexity and prevent overfitting by reducing feature map sizes early. Alice's smaller 25×25 images require smaller strides to preserve

sufficient spatial information, as aggressive downsampling could lose critical details. - **Kernel Size:** Bob is also more likely to use larger kernel sizes. Larger kernels capture broader contextual features, suitable for larger images where patterns (e.g., dog vs. cat features) may span more pixels. Alice's smaller images benefit from smaller kernels (e.g., 3×3) to focus on fine-grained details, as larger kernels might cover too much of the image, losing specificity.

Question 4: Autoencoders and Vision Transformers (15 points)

Part 1: Key idea behind Autoencoders and Vision Transformers (5+5=10 points)

- **Autoencoders:** Autoencoders are neural networks designed for unsupervised learning, consisting of an encoder and a decoder. The encoder compresses the input (e.g., an image) into a lower-dimensional latent representation, capturing essential features. The decoder reconstructs the input from this representation. For classification, the latent representation can be used as features for a classifier. The key idea is learning data representations by minimizing reconstruction loss. - **Vision Transformers (ViT):** ViTs adapt the Transformer architecture for images by dividing the image into fixed-size patches (e.g., 16×16 pixels), flattening each patch into a vector, and processing them as a sequence with self-attention mechanisms. The key idea is to model global relationships between patches using attention, enabling the model to capture long-range dependencies without convolutional operations.

Part 2: Choice for resource-constrained device (5 points)

For a resource-constrained device classifying medical images as benign or malignant, an **Autoencoder** is likely the better choice. Autoencoders are computationally lighter, as they involve simpler feedforward networks for encoding and decoding, and the latent representation can be used with a lightweight classifier. Vision Transformers require significant computational resources due to self-attention mechanisms, which scale poorly with sequence length (number of patches) and are memory-intensive, making them less suitable for resource-constrained environments.

Question 5: HoG Feature Descriptor (15 points)

Part 1: Full form and extraction process (10 points)

The full form of HoG is **Histogram of Oriented Gradients**.

HoG Feature Extraction Process: 1. **Preprocess Image:** Convert the image to grayscale and optionally normalize (e.g., gamma correction). 2. **Compute Gradients:** Calculate the gradient magnitude and orientation at each pixel using filters (e.g., Sobel). For pixel $I(x, y)$, gradients are $G_x = I(x+1, y) - I(x-1, y)$, $G_y = I(x, y+1) - I(x, y-1)$. Magnitude: $\sqrt{G_x^2 + G_y^2}$, Orientation: $\arctan(G_y/G_x)$. 3. **Divide into Cells:** Partition the image into small cells (e.g., 8×8 pixels). 4. **Build Histograms:** For each cell, create a histogram of gradient orientations, weighted by magnitude, over discrete bins (e.g., 9 bins for $0-180^\circ$). 5. **Block Normalization:** Group cells into larger blocks (e.g., 2×2 cells). Normalize gradient magnitudes within each block (e.g., L2-norm) to reduce illumination effects. 6. **Concatenate Features:** Collect normalized histograms from all

blocks to form the final HoG feature vector.

Part 2: Performance on textured vs. non-textured objects (5 points)

HoG performs better on **textured objects**. HoG relies on gradient orientations, which are rich in textured regions with varying intensities (e.g., edges, patterns). Textured objects, like a leopard's fur, yield more informative features due to distinct gradient patterns. Non-textured objects, like a smooth sphere, have fewer gradients, producing sparse histograms that are less discriminative. For example, in object recognition, a textured image of a dog's fur provides more robust HoG features than a non-textured image of a plain wall.

Question 6: Watershed-Based Segmentation (10 points)

Part 1: Key idea behind watershed-based segmentation (5 points)

The watershed algorithm treats an image as a topographic surface, where pixel intensities represent heights. The key idea is to simulate flooding from local minima (low-intensity regions), with "water" rising until boundaries (watershed lines) form between catchment basins, segmenting the image into regions. It is effective for separating touching objects or delineating regions with intensity gradients.

Part 2: Marker-based vs. non-marker-based methods (5 points)

- **Marker-Based Watershed:** Uses predefined markers (seeds) to indicate regions of interest (e.g., object centers). The algorithm floods from these markers, reducing over-segmentation by limiting the number of basins. It requires user input or preprocessing to place markers but produces more controlled results. - **Non-Marker-Based Watershed:** Floods from all local minima without predefined markers, relying solely on the image's intensity landscape. This can lead to over-segmentation, as every minor minimum creates a region, but it is fully automatic and requires no prior knowledge.

Question 7: SVM for Three-Class Classification (10 points)

The given linear SVM decision function is $f(x) = w^T x + b$, with classification rule: predict class +1 if $f(x) \geq 0$, else class -1, designed for binary classification.

Can it classify three classes (car, pedestrian, tree)?

No, the base SVM model cannot directly classify three classes, as it is designed for binary classification (two classes: +1 or -1). It separates data with a single hyperplane, which cannot partition the feature space into three distinct regions.

Strategy for three-class classification:

Use a **One-vs-Rest (OvR)** approach: - Train three binary SVM classifiers: - Classifier 1: Car (+1) vs. (Pedestrian, Tree) (-1). - Classifier 2: Pedestrian (+1) vs. (Car, Tree) (-1). - Classifier 3: Tree (+1) vs. (Car, Pedestrian) (-1). - For a new input x , compute decision functions $f_1(x)$, $f_2(x)$, $f_3(x)$ for each classifier. - Assign the class with the highest decision score: $\arg \max_i f_i(x)$.

This leverages the binary SVM model by training multiple classifiers, each distinguishing one class from the others, enabling multi-class classification.